DATA READINESS SYSTEM (DRS)

DETAILED DESIGN DOCUMENT (DDD)

Organization: The AI Institute Africa
Project: Data Readiness System (DRS)
Methodology: Hierarchical Model + V-Model
Document Type: Detailed Design Document (DDD)
Document Version: 1.0
Author: Wills (Project Lead)

1. INTRODUCTION

1.1 Purpose

This document specifies the detailed design of the Data Readiness System (DRS), including:

- Component-level architecture
- Module responsibilities
- Data structures and schemas
- Interface specifications
- Internal workflows
- Processing pipelines
- Deployment architecture
- Security and governance controls
- Testability and verification mapping

It forms the technical blueprint for implementation.

1.2 Scope

This DDD covers the full Data Readiness System including:

- Physical archive digitization pipeline
- Digital ingestion pipeline
- Document intelligence processing
- Trust scoring and validation workflow
- Canonical data model mapping
- Entity resolution and knowledge graph generation
- Dataset publishing and export
- Governance and compliance
- Platform operations and multi-tenancy

1.3 References

- Conceptual Design Document (CDD)
- Business Requirements Specification (BRS)
- System Requirements Specification (SRS) (to be produced)
- ISO/IEC 15288 Systems Engineering Lifecycle
- ISO 27001 Information Security Management
- GDPR / Data Protection standards (general compliance model)

1.4 Definitions and Acronyms

Term	Meaning
DRS	Data Readiness System
CDM	Common Data Model
OCR	Optical Character Recognition
RBAC	Role Based Access Control
HITL	Human In The Loop
MDM	Master Data Management
PII	Personally Identifiable Information
DDD	Detailed Design Document
V&V	Verification and Validation

2. SYSTEM DESIGN OVERVIEW

2.1 Design Goals

The system shall satisfy the following design goals:

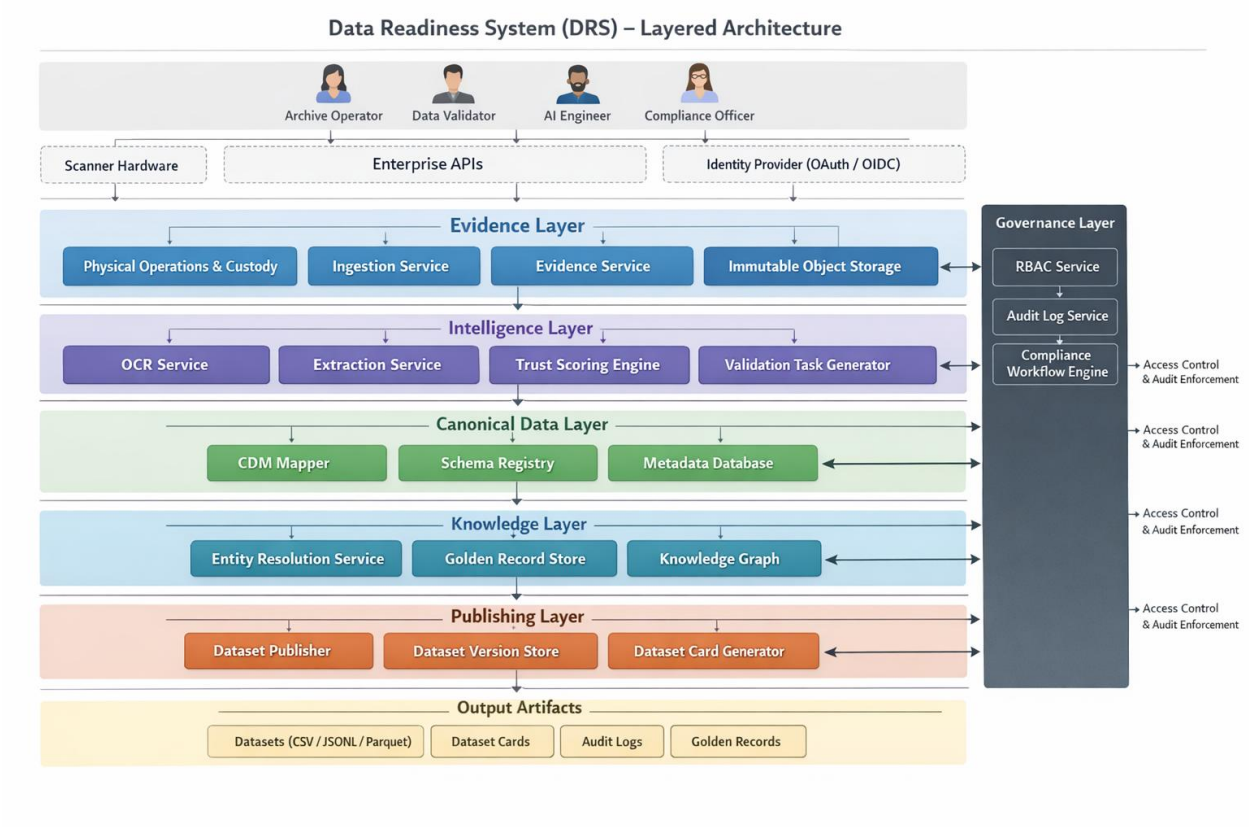
1. **End-to-end traceability** from dataset to evidence source.
2. **Immutable evidence preservation**.
3. **Human validation capability** for sensitive datasets.
4. **Scalable processing** of large document volumes.
5. **Governed dataset publishing** with versioning.
6. **Entity resolution** and golden record generation.
7. **Multi-tenant support** for multiple organizations.
8. **On-prem and air-gapped deployment compatibility**.

2.2 Architectural Pattern

The DRS is implemented as a **layered, service-oriented pipeline architecture**.

Layers

1. Evidence Layer
2. Intelligence Layer
3. Canonical Data Layer
4. Knowledge Layer
5. Publishing Layer
6. Governance Layer



2.3 High-Level Component Map

Input Sources

- Physical paper documents (scanned)
- Cloud drives
- Emails
- SharePoint repositories
- ERP/CRM exports

- Database dumps

Output Targets

- Dataset exports (CSV, JSONL, Parquet)
- Knowledge graph exports
- Embedding datasets
- Dataset cards
- Audit logs
- Golden record registry

3. DEPLOYMENT ARCHITECTURE (PHYSICAL DESIGN)

3.1 Supported Deployment Models

3.1.1 SaaS Cloud Deployment

- Hosted DRS platform
- Multi-tenant shared infrastructure
- Tenant isolation via logical separation

3.1.2 On-Prem Deployment

- Installed in organization data center
- Integrates with internal identity provider
- Local storage and compute

3.1.3 Air-Gapped Deployment

- No external internet connectivity
- Offline AI models
- Manual model update process

3.2 Recommended Infrastructure Stack

Compute

- Kubernetes cluster or Docker Swarm
- GPU nodes for OCR + LLM inference

- CPU nodes for orchestration and API services

Storage

- Object storage (S3-compatible)
- Relational database (PostgreSQL)
- Graph database (Neo4j / Amazon Neptune / ArangoDB)
- Search engine (OpenSearch / Elasticsearch)

Messaging

- Kafka / RabbitMQ
- Background task engine (Celery / Temporal)

3.3 Network Architecture

Network Zones

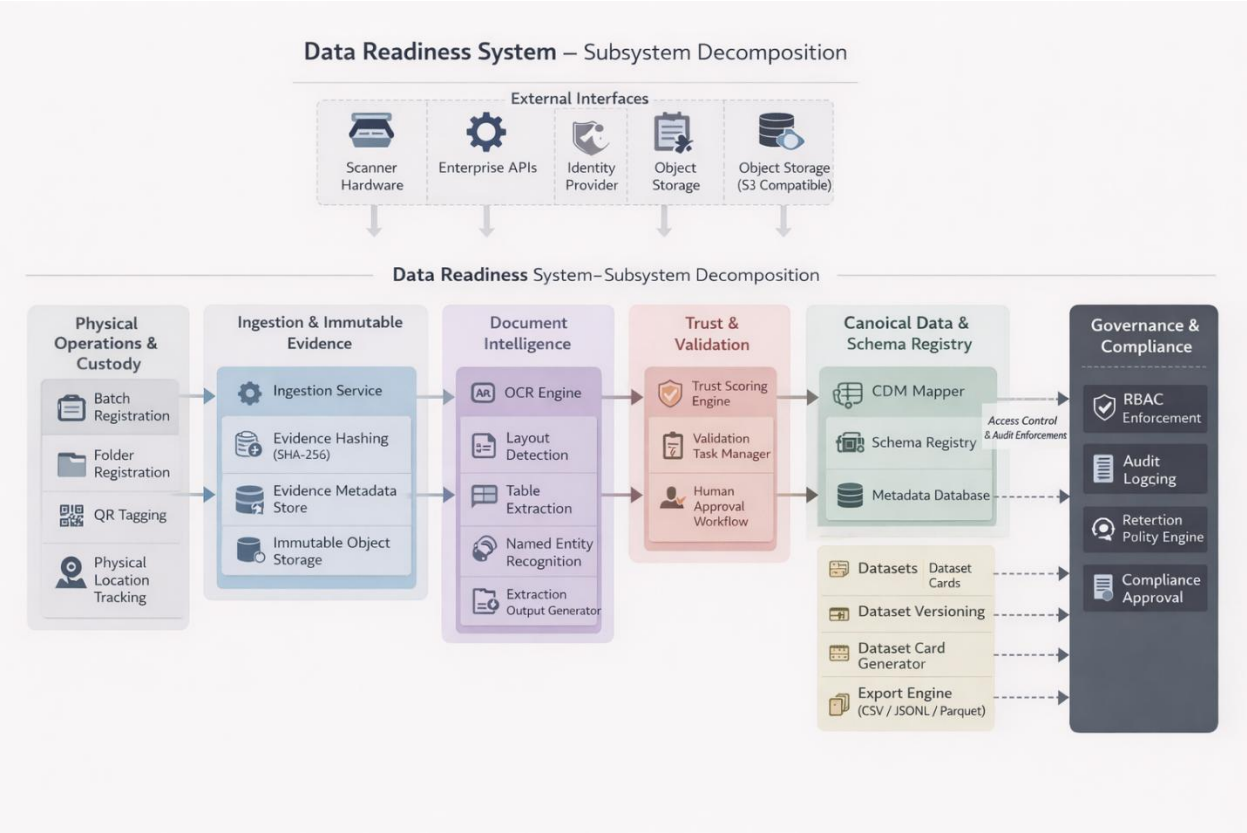
1. **External Zone**
    - Ingestion connectors
    - User portal access
2. **Processing Zone**
    - AI extraction services
    - scoring engines
3. **Data Zone**
    - databases
    - object storage
4. **Secure Governance Zone**
    - audit logs
    - key vault
    - access control engine

3.4 Recommended Services Layout

SERVICE	FUNCTION
GATEWAY API	Entry point
AUTH SERVICE	Authentication / RBAC
INGESTION SERVICE	File ingestion
EVIDENCE SERVICE	Immutable storage control
OCR SERVICE	OCR / handwriting
EXTRACTION SERVICE	Entity/table extraction
SCORING SERVICE	Trust scoring
VALIDATION SERVICE	HITL workflow

CDM MAPPER	Canonical mapping
ENTITY RESOLUTION SERVICE	Deduplication + merging
KNOWLEDGE GRAPH SERVICE	Graph creation
DATASET PUBLISHER	Dataset generation/export
AUDIT SERVICE	Logs and traceability
NOTIFICATION SERVICE	Workflow alerts

5. INTERNAL SYSTEM MODULE DESIGN (LOGICAL DESIGN)



4.1 Subsystem 1: Physical Operations & Custody

4.1.1 Responsibilities

- Manage physical document intake
- Assign batch IDs and QR codes
- Track custody chain
- Ensure scanning completeness

4.1.2 Physical Workflow Steps

1. Register batch
2. Register folders
3. Tag documents with QR
4. Scan pages
5. Perform image QA
6. Reconcile counts
7. Store physical records and log location

4.1.3 Data Structures

Batch Object

```
{
  "batch_id": "BATCH-2026-0001",
  "created_by": "operator_id",
  "created_at": "timestamp",
  "source_location": "Ministry Warehouse A",
  "status": "IN_PROGRESS",
  "expected_documents": 120,
  "scanned_documents": 98
}
```

Folder Object

```
{
  "folder_id": "FOLDER-00023",
  "batch_id": "BATCH-2026-0001",
  "qr_code": "QR_HASH_STRING",
  "physical_label": "Finance-2019-Q4",
  "status": "SCANNED"
}
```

Physical Location Object

```
{
  "box_id": "BOX-12",
  "shelf_id": "SHELF-3",
  "room_id": "ARCHIVE-ROOM-A",
  "building": "HQ"
}
```

4.1.4 Interfaces

- Scanner Interface → sends scan image files into Evidence Layer
- Custody Interface → writes custody logs into audit store

4.2 Subsystem 2: Ingestion & Immutable Evidence

4.2.1 Responsibilities

- Ingest all scanned and digital files
- Generate cryptographic hash
- Store immutable evidence
- Preserve provenance metadata

4.2.2 Evidence Storage Design

Object Storage Rules

- Evidence objects stored under a content-addressable path:

```
/evidence/{tenant_id}/{year}/{month}/{hash}/original.pdf
/evidence/{tenant_id}/{year}/{month}/{hash}/scan_page_001.png
```

Hashing

- Use SHA-256 for immutability validation

4.2.3 Evidence Object Metadata

```
{
  "evidence_id": "EVID-98A2",
  "tenant_id": "TENANT-001",
  "source_type": "SCAN",
  "source_reference": "BATCH-2026-0001/FOLDER-00023",
  "file_hash": "sha256:ab234...",
  "file_format": "pdf",
  "ingestion_timestamp": "2026-02-16T12:00:00Z",
  "stored_uri": "s3://evidence/.../original.pdf",
  "immutability_status": "LOCKED"
}
```

4.2.4 Digital Ingestion Connectors

Supported connector types:

- SharePoint Connector

- Google Drive Connector
- Email Connector
- FTP Connector
- Database Export Connector
- ERP/CRM Connector

Connector standard output is a **Standard Record Object (SRO)**.

4.3 Subsystem 3: Document Intelligence

4.3.1 Responsibilities

- Document classification
- Layout detection
- OCR
- Table extraction
- Entity extraction
- Produce structured candidate outputs

4.3.2 Document Classification Design

Classification outputs:

- Document type label
- Confidence score
- Suggested schema mapping

Example:

```
{
  "doc_id": "DOC-111",
  "doc_type": "Invoice",
  "confidence": 0.91
}
```

4.3.3 OCR Engine Design

OCR Pipeline

1. Image enhancement
2. OCR extraction

3. Language detection
4. Paragraph structuring
5. Token confidence scoring

OCR output structure:

```
{
  "page_id": "PAGE-001",
  "text_blocks": [
    {"text": "Invoice No: 3312", "bbox": [0,0,120,20], "confidence": 0.92}
  ]
}
```

4.3.4 Table Extraction Design

Table parser output:

```
{
  "table_id": "TABLE-001",
  "rows": [
    ["Item", "Qty", "Cost"],
    ["Laptop", "2", "1000"]
  ],
  "confidence": 0.87
}
```

4.3.5 Named Entity Extraction (NER)

Entities extracted:

- Names
- IDs
- Addresses
- Organizations
- Dates
- Transaction amounts
- Vehicle/asset identifiers

Entity output:

```
{
  "entity": "Company Name",
  "value": "ABC Holdings",
  "confidence": 0.94,
  "evidence_pointer": "PAGE-001:bbox(10,20,80,40)"
}
```

4.4 Subsystem 4: Trust & Validation

4.4.1 Responsibilities

- Evaluate extraction trustworthiness
- Detect anomalies and duplicates
- Trigger human validation workflows
- Approve or reject extracted values

4.4.2 Trust Scoring Model

Trust score = weighted sum of:

- OCR confidence
- Extraction confidence
- Document quality score
- Completeness score
- Cross-source consistency
- Validator approval history

Example trust scoring formula:

```
TrustScore = 0.35*OCR + 0.25*Extraction + 0.15*Completeness +
             0.15*Consistency + 0.10*DocQuality
```

4.4.3 Validation Workflow Design (HITL)

Workflow states:

- PENDING_EXTRACTION
- PENDING_VALIDATION
- APPROVED
- REJECTED
- NEEDS_RESCAN
- ESCALATED

Validation object:

```
{
  "validation_id": "VAL-002",
  "record_object_id": "SRO-919",
  "assigned_to": "validator_007",
  "status": "PENDING_VALIDATION",
  "fields_to_validate": ["invoice_number", "amount", "supplier_name"]
```

}

4.4.4 Multi-Approver Design

For sensitive datasets:

- 2-step or 3-step approval process
- compliance officer final signoff

4.5 Subsystem 5: Canonical Data & Schema Registry

4.5.1 Responsibilities

- Define CDM entities and relationships
- Version schemas
- Enforce structured mapping rules
- Support tenant-specific extensions

4.5.2 Common Data Model (CDM)

Core entities:

Person

- person_id
- name
- national_id
- birth_date
- phone
- address
- email

Organization

- org_id
- org_name
- registration_number
- sector
- address

Document

- doc_id
- doc_type
- evidence_id
- created_date
- source_system

Transaction

- txn_id
- txn_type
- amount
- currency
- date
- payer_id
- payee_id

Asset

- asset_id
- asset_type
- serial_number
- ownership_status

4.5.3 Schema Registry Design

Schema version format:

- vMajor.Minor.Patch
- Example: v2.1.0

Schema storage includes:

- schema_id
- schema_json
- created_by
- timestamp
- migration scripts

4.6 Subsystem 6: Knowledge Synchronization & Entity Resolution

4.6.1 Responsibilities

- Deduplicate records
- Merge into golden records
- Build knowledge graph relationships
- Synchronize with enterprise systems

4.6.2 Entity Resolution Design

Matching criteria:

- Exact match (ID number, passport number)
- Fuzzy match (names, addresses)
- Graph-based match (shared documents, shared phone numbers)

Resolution outputs:

- MATCHED
- POSSIBLE_MATCH
- NO_MATCH
- NEEDS_HUMAN_REVIEW

Example:

```
{
  "candidate_a": "PERSON-001",
  "candidate_b": "PERSON-102",
  "match_score": 0.89,
  "status": "POSSIBLE_MATCH"
}
```

4.6.3 Golden Record Merge Rules

Rules:

- ID number is authoritative if verified
- Most recent validated address is selected
- Multiple phone numbers can coexist

Golden record object:

```
{
  "golden_person_id": "GP-001",
  "merged_from": ["PERSON-001", "PERSON-102"],
  "fields": {
```

```
    "name": "John Doe",
    "national_id": "63-111111A",
    "address": "Harare"
  }
}
```

4.6.4 Knowledge Graph Model

Graph nodes:

- Person
- Organization
- Document
- Asset
- Transaction
- Case

Graph edges:

- OWNS
- WORKS_FOR
- PAID_TO
- SIGNED
- REGISTERED_BY
- LOCATED_AT

Graph storage:

- Neo4j recommended for MVP

4.7 Subsystem 7: Dataset Publishing

4.7.1 Responsibilities

- Generate datasets from CDM and knowledge graph
- Version datasets
- Produce dataset cards
- Export in multiple formats

4.7.2 Dataset Versioning Strategy

Dataset version includes:

- dataset_id
- schema_version
- extraction_model_version
- timestamp
- approval signature

Example:

```
{
  "dataset_id": "DATASET-FINANCE-2026",
  "version": "v1.0.0",
  "generated_at": "2026-02-16T13:00:00Z",
  "source_batches": ["BATCH-2026-0001"],
  "schema_version": "v2.1.0"
}
```

4.7.3 Dataset Card Format

Dataset card includes:

- dataset name
- purpose
- schema
- lineage
- quality metrics
- PII classification
- retention policy
- validation signoff

4.8 Subsystem 8: Governance & Compliance

4.8.1 Responsibilities

- enforce RBAC
- manage audit logs
- handle PII
- retention policies
- compliance approvals

4.8.2 RBAC Model

Roles:

- Operator
- Validator
- Data Engineer
- AI Engineer
- Compliance Officer
- Admin

Permissions:

- ingest evidence
- view evidence
- approve validation
- export datasets
- delete/retire dataset versions (if allowed)
- view audit logs

4.8.3 Audit Logging Design

Audit event includes:

- who
- action
- timestamp
- object affected
- before/after values
- reason

Example:

```
{
  "event_id": "AUD-778",
  "actor": "validator_007",
  "action": "APPROVE_FIELD",
  "object": "SRO-919.amount",
  "timestamp": "2026-02-16T14:22:00Z"
}
```

4.8.4 PII Detection and Redaction

PII categories:

- National IDs

- passports
- phone numbers
- email addresses
- addresses
- financial identifiers

Redaction modes:

- mask (****)
- pseudonymization
- tokenization

4.9 Subsystem 9: Platform Operations

4.9.1 Responsibilities

- multi-tenancy isolation
- monitoring and observability
- backups
- failover
- model version rollback

4.9.2 Multi-Tenant Design

Tenant isolation must apply to:

- object storage prefix
- database schema namespace
- graph namespace
- audit logs separation

4.9.3 Monitoring Design

Key metrics:

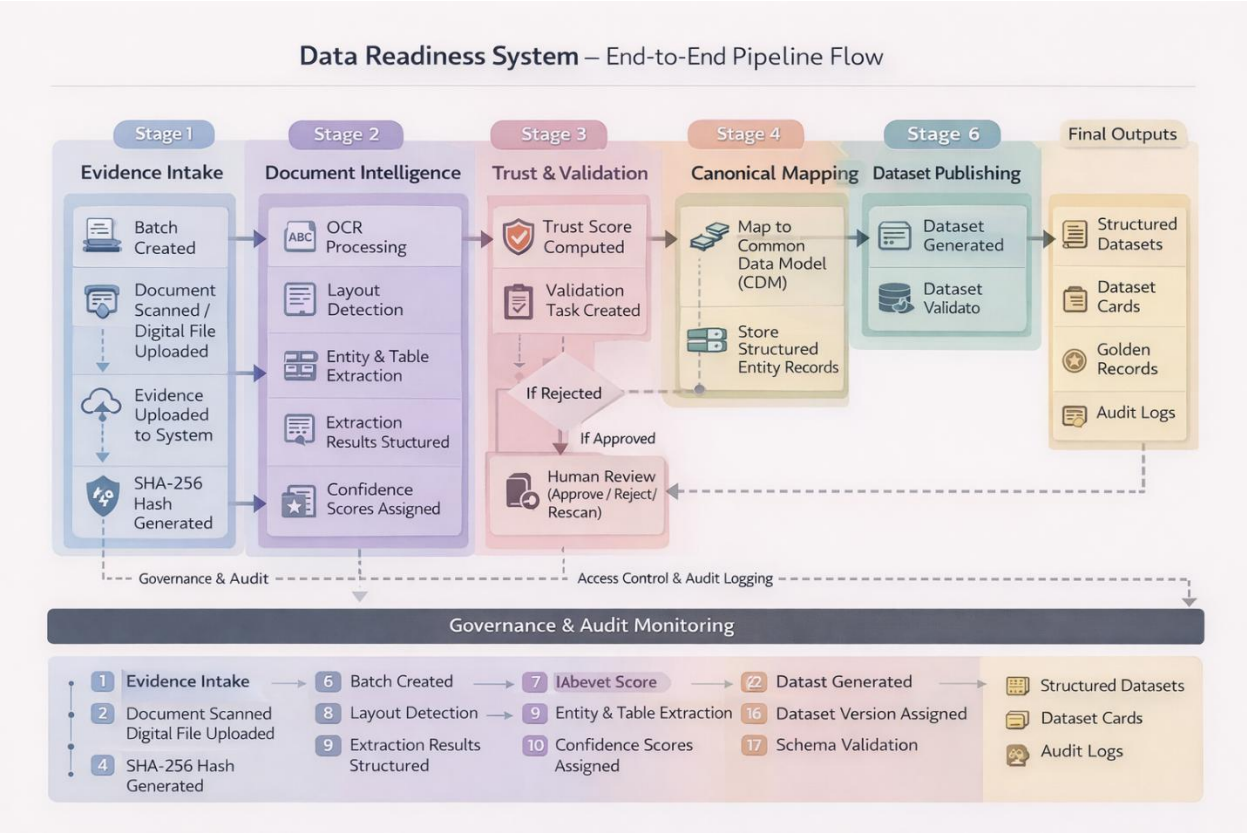
- ingestion throughput
- OCR processing latency
- extraction accuracy
- validation queue backlog

- dataset generation time
- storage growth

Tools:

- Prometheus + Grafana
- ELK / OpenSearch logs

5. END-TO-END PIPELINE DETAILED DESIGN



5.1 Physical Document Pipeline

Step-by-step

1. Batch created
2. QR assigned
3. Document scanned
4. Scan stored in evidence store
5. OCR performed
6. Extraction candidates created
7. Trust scoring performed
8. Validation triggered

9. Approved values mapped into CDM
10. Entity resolution performed
11. Knowledge graph updated
12. Dataset published

5.2 Digital Pipeline

Flow:
Connector → Evidence Store → Record Object → Extraction → Validation → CDM → Entity Resolution → Golden Record → Knowledge Graph → Dataset Publishing

6. DATABASE DESIGN

6.1 Recommended Database Choices

Data Type	Storage
Metadata	PostgreSQL
Evidence files	Object Storage
Search indexes	OpenSearch
Graph relationships	Neo4j

6.2 Core Tables (Relational)

- tenants
- users
- roles
- batches
- folders
- documents
- evidence_objects
- record_objects
- extraction_results
- validation_tasks
- cdm_entities
- entity_links
- dataset_versions
- audit_events

7. API DESIGN (INTERFACE CONTROL DOCUMENT STYLE)

7.1 API Gateway Endpoints

Ingestion APIs

- POST /ingestion/upload
- POST /ingestion/connector/start
- GET /ingestion/status/{id}

Evidence APIs

- GET /evidence/{id}
- GET /evidence/{id}/metadata

Extraction APIs

- POST /intelligence/extract/{doc_id}

Validation APIs

- GET /validation/tasks
- POST /validation/approve
- POST /validation/reject

Publishing APIs

- POST /publish/dataset
- GET /publish/dataset/{id}/download

8. SECURITY DESIGN

8.1 Authentication

- OAuth2 / OpenID Connect
- MFA support

8.2 Encryption

- AES-256 at rest

- TLS 1.2+ in transit

8.3 Key Management

- Vault / KMS recommended
- rotated keys

9. FAILURE HANDLING AND RECOVERY

9.1 Retry Policy

- evidence ingestion retries: 3
- OCR retries: 2
- extraction retries: 2
- publishing retries: 3

9.2 Idempotency

All ingestion operations must be idempotent using evidence hash keys.

10. TEST DESIGN (V-MODEL SUPPORT)

10.1 Unit Testing

- hash generation tests
- OCR accuracy tests
- schema validation tests

10.2 Integration Testing

- ingestion → extraction → validation
- validation → CDM mapping
- CDM → entity resolution

10.3 System Testing

- full batch digitization test
- full dataset publishing test

10.4 Acceptance Testing

- traceability test (dataset → evidence pointer)
- governance compliance test
- air-gapped deployment test

11. IMPLEMENTATION ROADMAP (PHASED DELIVERY)

Phase 1: MVP

- Evidence store
- OCR + extraction
- validation workflow
- CDM mapping
- CSV dataset publishing

Phase 2: Knowledge Graph + Entity Resolution

- golden record engine
- graph sync
- advanced lineage

Phase 3: Full Governance + Multi-tenancy

- RBAC hardened
- compliance approvals
- air-gapped deployment

Phase 4: Advanced AI Dataset Factory

- embedding exports
- model retraining loops
- active learning validation prioritization

12. CONCLUSION

This detailed design defines all modules, interfaces, storage structures, workflows, governance controls, and deployment requirements needed for the Data Readiness System to operate as a legally defensible AI-ready dataset factory.